

LAB level 3 PROJECT

EcoFoodSync: Polyglot Database System for Automated Perishable Food Rescue

DBMS Engine: MySQL 8.0

Course: DBMS Experiential Learning – Level 3

Lab Task: ER Diagram & DDL Schema Implementation

Team : G14

SRN1: PES2UG24CS660

SRN2: PES2UG24CS676

SECTION

1. Lab Objective

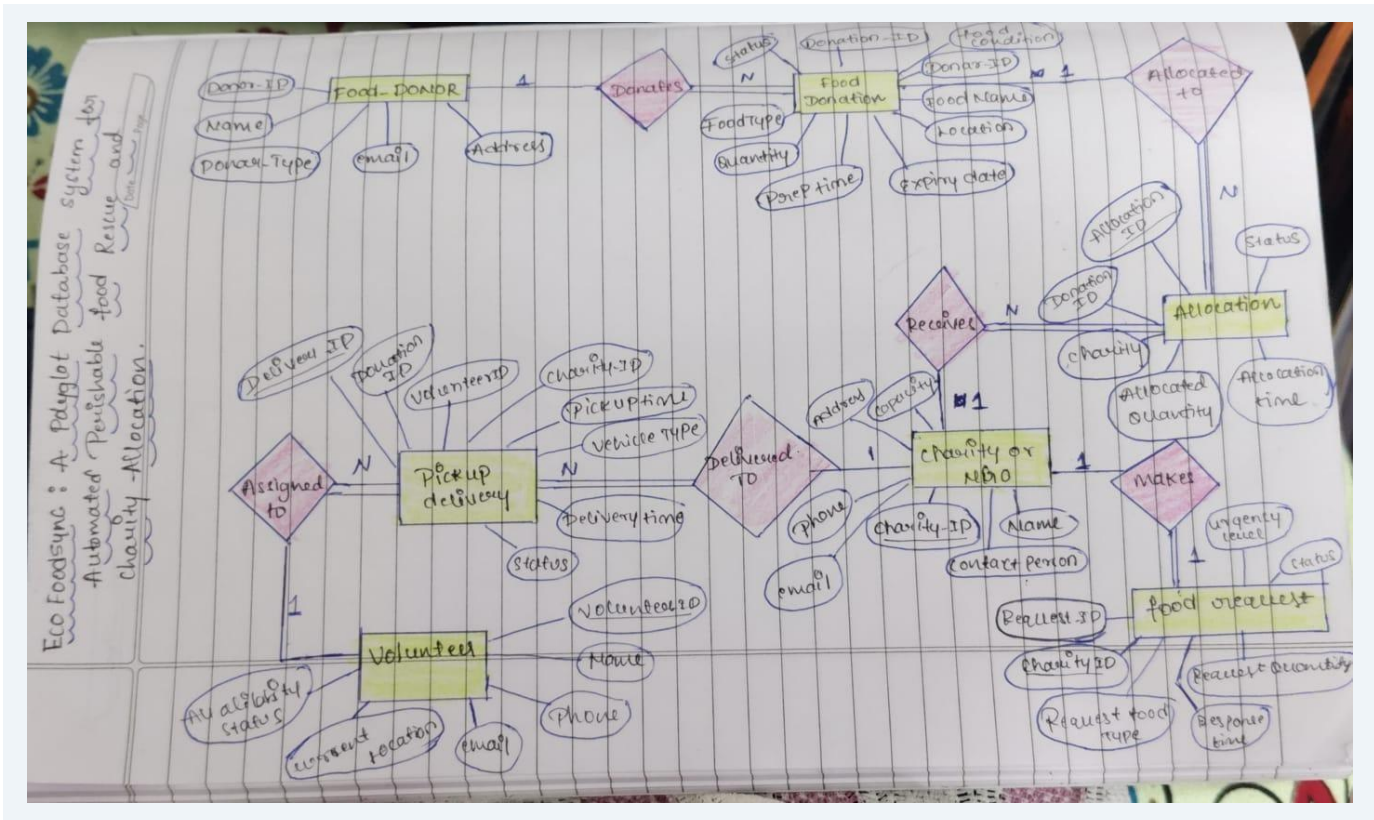
To design a normalised relational schema, define logical entities, construct an Entity-Relationship (ER) Diagram using standard Chen notation, and implement database structures using Data Definition Language (DDL) commands in MySQL 8.0 with primary keys, foreign keys, constraints, and performance indexes.

2. System Architecture & Core Entities

The EcoFoodSync system coordinates perishable food rescue across 7 key relational entities:

- **FOOD_DONOR:** Stores donor business profiles (Restaurants, Supermarkets, Caterers, Individuals).
- **FOOD_DONATION:** Tracks surplus food batches, preparation/expiry timestamps, and condition types.
- **CHARITY_OR_NGO:** Manages recipient organisations, contact details, operational areas, and food capacity.
- **ALLOCATION:** Maps food donations to charities while enforcing allocation status and quantities.
- **VOLUNTEER:** Stores driver information, real-time availability status, and current locations.
- **PICKUP_DELIVERY:** Handles physical transportation and logistics logs from donor to recipient charity.
- **FOOD_REQUEST:** Tracks specific food demand requests submitted by charities based on urgency level.

3. Entity-Relationship (ER) Diagram (Chen Notation)



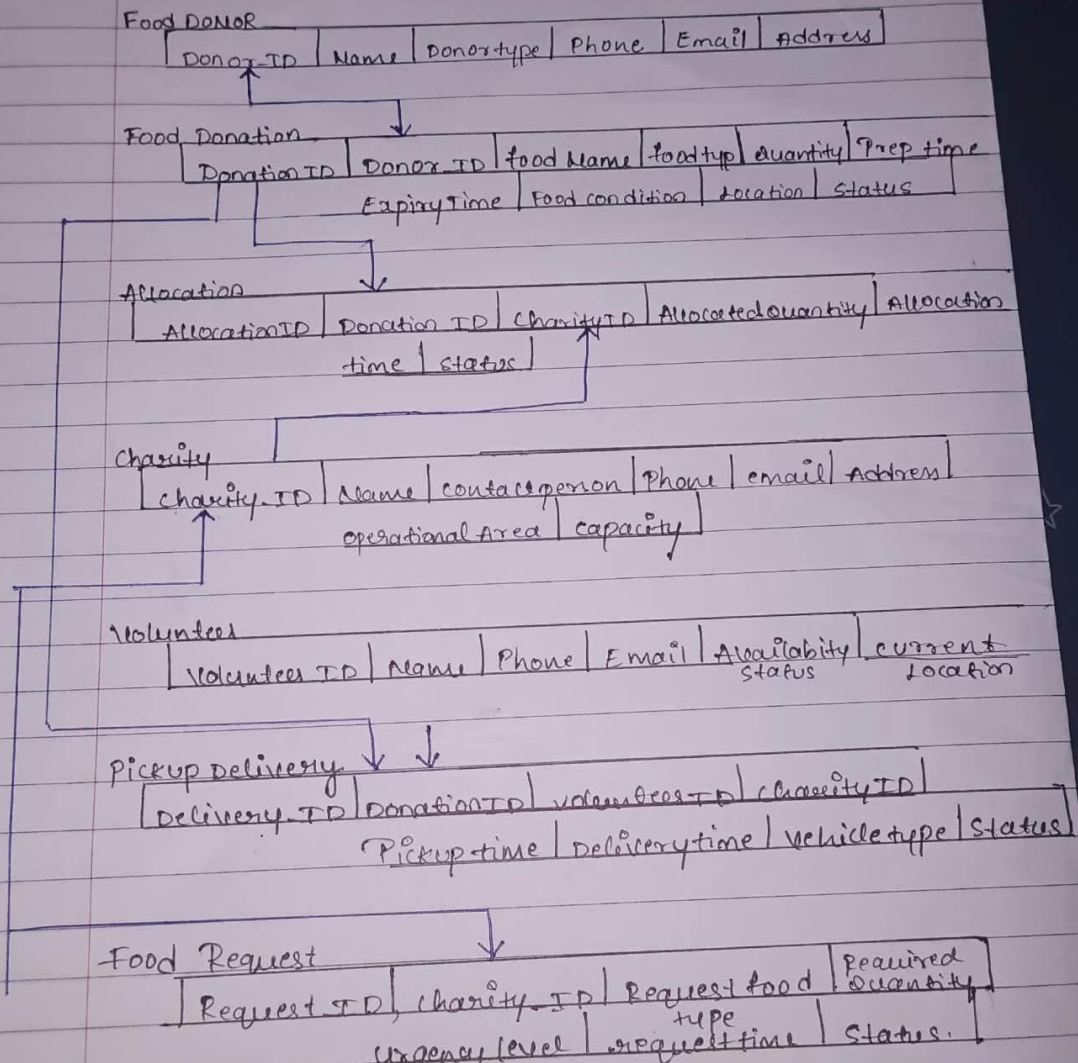
Cardinality & Relationship Summary

- **FOOD_DONOR (1) : DONATES : FOOD_DONATION (N)** — One donor can generate multiple food donation listings.
- **FOOD_DONATION (1) : ALLOCATED TO : ALLOCATION (M)** — One donation batch can be split into multiple allocations.
- **ALLOCATION (N) : RECEIVES ALLOCATION : CHARITY_OR_NGO (1)** — One charity can receive multiple allocations.
- **CHARITY_OR_NGO (1) : MAKES REQUEST : FOOD_REQUEST (N)** — One charity can issue multiple food requests.
- **VOLUNTEER (1) : ASSIGNED TO : PICKUP_DELIVERY (N)** — One volunteer driver handles multiple delivery tasks.
- **FOOD_DONATION (1) : PICKED UP / DELIVERED : PICKUP_DELIVERY (N)** — One donation batch links to logistics tasks.
- **PICKUP_DELIVERY (N) : DELIVERED TO : CHARITY_OR_NGO (1)** — Multiple delivery actions arrive at a single target charity.

SCHEMA DIAGRAM

EcoFood sync: A Polysql Database system for Automated Perishable food rescue and charity Allocation

Schema model



4. Data Dictionary

Table Name	Attribute Name	Data Type	Constraint	Description
food_donor	donor_id	CHAR(36)	PRIMARY KEY	UUID string generated via DEFAULT (UUID())
food_donor	name	VARCHAR(255)	NOT NULL	Legal name of donor entity
food_donor	donor_type	ENUM(...)	NOT NULL	Type (RESTAURANT, SUPERMARKET, etc.)
food_donor	email	VARCHAR(255)	UNIQUE, NOT NULL	Unique donor contact email

food_donation	donation_id	CHAR(36)	PRIMARY KEY	Unique food donation batch ID
food_donation	donor_id	CHAR(36)	FOREIGN KEY	FK referencing food_donor(donor_id)
food_donation	quantity	DECIMAL(10,2)	CHECK (>0)	Total quantity/weight in KG
food_donation	expiry_time	DATETIME	CHECK (>prep)	Timestamp before which food expires
allocation	allocation_id	CHAR(36)	PRIMARY KEY	Unique allocation claim ID
allocation	donation_id	CHAR(36)	FOREIGN KEY	FK referencing food_donation
allocation	charity_id	CHAR(36)	FOREIGN KEY	FK referencing charity_or_ngo
pickup_delivery	delivery_id	CHAR(36)	PRIMARY KEY	Unique logistics tracking ID
pickup_delivery	volunteer_id	CHAR(36)	FOREIGN KEY	FK referencing volunteer (NULL allowed)

5. MySQL 8.0 DDL Implementation Script

Create :

-- Database Creation

```
mysql> CREATE DATABASE IF NOT EXISTS ecofoodsync_db;
Query OK, 1 row affected (0.19 sec)

mysql> USE ecofoodsync_db;
Database changed
```

-- 1. FOOD_DONOR Table

```
mysql> CREATE TABLE food_donor (
->   donor_id CHAR(36) PRIMARY KEY DEFAULT (UUID()),
->   name VARCHAR(255) NOT NULL,
->   donor_type ENUM('RESTAURANT', 'SUPERMARKET', 'INDIVIDUAL', 'CATERER', 'OTHER') NOT NULL,
->   phone VARCHAR(20) NOT NULL,
->   email VARCHAR(255) NOT NULL,
->   address TEXT NOT NULL,
->   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->   CONSTRAINT uk_food_donor_email UNIQUE (email)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.21 sec)
```

```
mysql> SHOW COLUMNS FROM food_donor;
```

Field	Type	Null	Key	Default	Extra
donor_id	char(36)	NO	PRI	uuid()	DEFAULT_GENERATED
name	varchar(255)	NO		NULL	
donor_type	enum('RESTAURANT', 'SUPERMARKET', 'INDIVIDUAL', 'CATERER', 'OTHER')	NO		NULL	
phone	varchar(20)	NO		NULL	
email	varchar(255)	NO	UNI	NULL	
address	text	NO		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
7 rows in set (0.00 sec)
```

```
-- 2. FOOD_DONATION Table
```

```
mysql> CREATE TABLE food_donation (
->   donation_id CHAR(36) PRIMARY KEY DEFAULT (UUID()),
->   donor_id CHAR(36) NOT NULL,
->   food_name VARCHAR(255) NOT NULL,
->   food_type VARCHAR(100) NOT NULL,
->   quantity DECIMAL(10, 2) NOT NULL,
->   preparation_time DATETIME NOT NULL,
->   expiry_time DATETIME NOT NULL,
->   food_condition ENUM('FRESH', 'PACKAGED', 'PERISHABLE_FAST', 'COOKED') NOT NULL,
->   location TEXT NOT NULL,
->   status VARCHAR(50) DEFAULT 'AVAILABLE',
->   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->   CONSTRAINT fk_donation_donor FOREIGN KEY (donor_id) REFERENCES food_donor(donor_id) ON DELETE CASCADE,
->   CONSTRAINT chk_quantity_positive CHECK (quantity > 0),
->   CONSTRAINT chk_expiry_after_prep CHECK (expiry_time > preparation_time)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.15 sec)
```

```
mysql> describe food_donation;
```

Field	Type	Null	Key	Default	Extra
donation_id	char(36)	NO	PRI	uuid()	DEFAULT_GENERATED
donor_id	char(36)	NO	MUL	NULL	
food_name	varchar(255)	NO	MUL	NULL	
food_type	varchar(100)	NO		NULL	
quantity	decimal(10,2)	NO		NULL	
preparation_time	datetime	NO		NULL	
expiry_time	datetime	NO		NULL	
food_condition	enum('FRESH', 'PACKAGED', 'PERISHABLE_FAST', 'COOKED')	NO		NULL	
location	text	NO		NULL	
status	varchar(50)	YES	MUL	AVAILABLE	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
11 rows in set (0.00 sec)
```

```
-- 3. CHARITY_OR_NGO Table
```

```
mysql> CREATE TABLE charity_or_ngo (
->   charity_id CHAR(36) PRIMARY KEY DEFAULT (UUID()),
->   name VARCHAR(255) NOT NULL,
->   contact_person VARCHAR(150) NOT NULL,
->   phone VARCHAR(20) NOT NULL,
->   email VARCHAR(255) NOT NULL,
->   address TEXT NOT NULL,
->   operational_area VARCHAR(150) NOT NULL,
->   capacity INT NOT NULL,
->   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->   CONSTRAINT uk_charity_email UNIQUE (email),
->   CONSTRAINT chk_capacity_positive CHECK (capacity > 0)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.13 sec)
```

```
mysql> SHOW COLUMNS FROM charity_or_ngo;
```

Field	Type	Null	Key	Default	Extra
charity_id	char(36)	NO	PRI	uuid()	DEFAULT_GENERATED
name	varchar(255)	NO		NULL	
contact_person	varchar(150)	NO		NULL	
phone	varchar(20)	NO		NULL	
email	varchar(255)	NO	UNI	NULL	
address	text	NO		NULL	
operational_area	varchar(150)	NO		NULL	
capacity	int	NO		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
9 rows in set (0.00 sec)
```

```
-- 4. ALLOCATION Table
```

```
mysql> CREATE TABLE allocation (
->     allocation_id CHAR(36) PRIMARY KEY DEFAULT (UUID()),
->     donation_id CHAR(36) NOT NULL,
->     charity_id CHAR(36) NOT NULL,
->     allocated_quantity DECIMAL(10, 2) NOT NULL,
->     allocation_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->     status ENUM('PENDING', 'APPROVED', 'REJECTED', 'COMPLETED') DEFAULT 'PENDING',
->     CONSTRAINT fk_allocation_donation FOREIGN KEY (donation_id) REFERENCES food_donation(donation_id) ON D
ELETE CASCADE,
->     CONSTRAINT fk_allocation_charity FOREIGN KEY (charity_id) REFERENCES charity_or_ngo(charity_id) ON DEL
ETE CASCADE,
->     CONSTRAINT chk_allocated_qty_positive CHECK (allocated_quantity > 0)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.16 sec)
```

```
mysql> SHOW COLUMNS FROM allocation;
```

Field	Type	Null	Key	Default	Extra
allocation_id	char(36)	NO	PRI	uuid()	DEFAULT_GENERATED
donation_id	char(36)	NO	MUL	NULL	
charity_id	char(36)	NO	MUL	NULL	
allocated_quantity	decimal(10,2)	NO		NULL	
allocation_time	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
status	enum('PENDING', 'APPROVED', 'REJECTED', 'COMPLETED')	YES		PENDING	

```
6 rows in set (0.01 sec)
```

```
-- 5. VOLUNTEER Table
```

```
mysql> CREATE TABLE volunteer (
->     volunteer_id CHAR(36) PRIMARY KEY DEFAULT (UUID()),
->     name VARCHAR(255) NOT NULL,
->     phone VARCHAR(20) NOT NULL,
->     email VARCHAR(255) NOT NULL,
->     availability_status BOOLEAN DEFAULT TRUE,
->     current_location TEXT,
->     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->     CONSTRAINT uk_volunteer_email UNIQUE (email)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.12 sec)
```

```
mysql> SHOW COLUMNS FROM volunteer;
```

Field	Type	Null	Key	Default	Extra
volunteer_id	char(36)	NO	PRI	uuid()	DEFAULT_GENERATED
name	varchar(255)	NO		NULL	
phone	varchar(30)	NO	UNI	NULL	
email	varchar(255)	NO	UNI	NULL	
availability_status	tinyint(1)	YES		1	
current_location	text	YES		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
7 rows in set (0.00 sec)
```

```
-- 6. PICKUP_DELIVERY Table
```

```
mysql> CREATE TABLE pickup_delivery (
->   delivery_id CHAR(36) PRIMARY KEY DEFAULT (UUID()),
->   donation_id CHAR(36) NOT NULL,
->   volunteer_id CHAR(36),
->   charity_id CHAR(36) NOT NULL,
->   pickup_time DATETIME,
->   delivery_time DATETIME,
->   vehicle_type VARCHAR(100),
->   status ENUM('ASSIGNED', 'PICKED_UP', 'IN_TRANSIT', 'DELIVERED', 'CANCELLED') DEFAULT 'ASSIGNED',
->   notes TEXT,
->   CONSTRAINT fk_delivery_donation FOREIGN KEY (donation_id) REFERENCES food_donation(donation_id) ON DELETE CASCADE,
->   CONSTRAINT fk_delivery_volunteer FOREIGN KEY (volunteer_id) REFERENCES volunteer(volunteer_id) ON DELETE SET NULL,
->   CONSTRAINT fk_delivery_charity FOREIGN KEY (charity_id) REFERENCES charity_or_ngo(charity_id) ON DELETE CASCADE
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.14 sec)
```

```
mysql> SHOW COLUMNS FROM pickup_delivery;
```

Field	Type	Null	Key	Default	Extra
delivery_id	char(36)	NO	PRI	uuid()	DEFAULT_GENERATED
donation_id	char(36)	NO	MUL	NULL	
volunteer_id	char(36)	YES	MUL	NULL	
charity_id	char(36)	NO	MUL	NULL	
pickup_time	datetime	YES		NULL	
delivery_time	datetime	YES		NULL	
vehicle_type	varchar(100)	YES		NULL	
status	enum('ASSIGNED', 'PICKED_UP', 'IN_TRANSIT', 'DELIVERED', 'CANCELLED')	YES		ASSIGNED	
delivery_notes	text	YES		NULL	

```
9 rows in set (0.00 sec)
```

-- 7. FOOD_REQUEST Table

```
mysql> CREATE TABLE food_request (
->   request_id CHAR(36) PRIMARY KEY DEFAULT (UUID()),
->   charity_id CHAR(36) NOT NULL,
->   required_food_type VARCHAR(100) NOT NULL,
->   required_quantity DECIMAL(10, 2) NOT NULL,
->   urgency_level ENUM('LOW', 'MEDIUM', 'HIGH', 'CRITICAL') DEFAULT 'MEDIUM',
->   request_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
->   status VARCHAR(50) DEFAULT 'OPEN',
->   CONSTRAINT fk_request_charity FOREIGN KEY (charity_id) REFERENCES charity_or_ngo(charity_id) ON DELETE CASCADE,
->   CONSTRAINT chk_required_qty_positive CHECK (required_quantity > 0)
-> ) ENGINE=InnoDB;
Query OK, 0 rows affected (0.13 sec)
```

```
mysql> SHOW COLUMNS FROM food_request;
```

Field	Type	Null	Key	Default	Extra
request_id	char(36)	NO	PRI	uuid()	DEFAULT_GENERATED
charity_id	char(36)	NO	MUL	NULL	
required_food_type	varchar(100)	NO		NULL	
required_quantity	decimal(10,2)	NO		NULL	
urgency_level	enum('LOW', 'MEDIUM', 'HIGH', 'CRITICAL')	YES		MEDIUM	
request_time	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED
status	varchar(50)	YES		OPEN	

```
7 rows in set (0.00 sec)
```

Indexing :

```
mysql>
mysql> CREATE INDEX idx_food_donation_donor ON food_donation(donor_id);
Query OK, 0 rows affected (0.09 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_allocation_donation ON allocation(donation_id);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_allocation_charity ON allocation(charity_id);
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_pickup_delivery_volunteer ON pickup_delivery(volunteer_id);
Query OK, 0 rows affected (0.07 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx_food_request_charity ON food_request(charity_id);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> SHOW INDEX FROM food_donation;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
food_donation	0	PRIMARY	1	donation_id	A	0	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_food_donation_donor	1	donor_id	A	0	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_donation_status_expiry	1	status	A	0	NULL	NULL	YES	BTREE			YES	NULL
food_donation	1	idx_donation_status_expiry	2	expiry_time	A	0	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_donation_status_expiry	3	food_condition	A	0	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_food_name_fulltext	1	food_name	NULL	0	NULL	NULL		FULLTEXT			YES	NULL

```
6 rows in set (0.07 sec)
```

-- 1. ALTER OPERATIONS

-- 1. Modify phone column in volunteer

```
mysql> ALTER TABLE volunteer
->      MODIFY COLUMN phone VARCHAR(30) NOT NULL;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> describe volunteer;
```

Field	Type	Null	Key	Default	Extra
volunteer_id	char(36)	NO	PRI	uuid()	DEFAULT_GENERATED
name	varchar(255)	NO		NULL	
phone	varchar(30)	NO	UNI	NULL	
email	varchar(255)	NO	UNI	NULL	
availability_status	tinyint(1)	YES		1	
current_location	text	YES		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
7 rows in set (0.00 sec)
```

-- 2. Rename notes column in pickup_delivery

```
mysql> ALTER TABLE pickup_delivery
->      RENAME COLUMN notes TO delivery_notes;
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> describe pickup_delivery;
```

Field	Type	Null	Key	Default	Extra
delivery_id	char(36)	NO	PRI	uuid()	DEFAULT_GENERATED
donation_id	char(36)	NO	MUL	NULL	
volunteer_id	char(36)	YES	MUL	NULL	
charity_id	char(36)	NO	MUL	NULL	
pickup_time	datetime	YES		NULL	
delivery_time	datetime	YES		NULL	
vehicle_type	varchar(100)	YES		NULL	
status	enum('ASSIGNED','PICKED_UP','IN_TRANSIT','DELIVERED','CANCELLED')	YES		ASSIGNED	
delivery_notes	text	YES		NULL	

```
9 rows in set (0.00 sec)
```

-- 3. DROP Operation (Demonstration)

```
mysql> CREATE TABLE IF NOT EXISTS temp_audit_logs (
  ->     log_id INT AUTO_INCREMENT PRIMARY KEY,
  ->     log_message VARCHAR(255)
  -> );
Query OK, 0 rows affected (0.08 sec)

mysql> DROP TABLE IF EXISTS temp_audit_logs;
Query OK, 0 rows affected (0.04 sec)
```

-- 4. Add Constraints

```
mysql> ALTER TABLE food_donor
  ->     ADD CONSTRAINT chk_donor_rating CHECK (donor_rating >= 0.00 AND donor_rating <= 5.00);
Query OK, 0 rows affected (0.22 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql>
mysql> ALTER TABLE volunteer
  ->     ADD CONSTRAINT uk_volunteer_phone UNIQUE (phone);
Query OK, 0 rows affected (0.05 sec)
Records: 0 Duplicates: 0 Warnings: 0
```

```
mysql> describe food_donor;
```

Field	Type	Null	Key	Default	Extra
donor_id	char(36)	NO	PRI	uuid()	DEFAULT_GENERATED
name	varchar(255)	NO		NULL	
donor_type	enum('RESTAURANT','SUPERMARKET','INDIVIDUAL','CATERER','OTHER')	NO		NULL	
phone	varchar(20)	NO		NULL	
email	varchar(255)	NO	UNI	NULL	
is_verified	tinyint(1)	YES		0	
donor_rating	decimal(3,2)	YES		5.00	
address	text	NO		NULL	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
9 rows in set (0.00 sec)
```

-- 5. Create Indexes

```
mysql>
mysql> CREATE INDEX idx_donation_status_expiry ON food_donation(status, expiry_time, food_condition);
Query OK, 0 rows affected (0.08 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE FULLTEXT INDEX idx_food_name_fulltext ON food_donation(food_name);
Query OK, 0 rows affected, 1 warning (0.48 sec)
Records: 0 Duplicates: 0 Warnings: 1

mysql> CREATE INDEX idx_pickup_active_deliveries ON pickup_delivery(volunteer_id, status);
Query OK, 0 rows affected (0.04 sec)
```

```
mysql> describe food_donation;
```

Field	Type	Null	Key	Default	Extra
donation_id	char(36)	NO	PRI	uuid()	DEFAULT_GENERATED
donor_id	char(36)	NO	MUL	NULL	
food_name	varchar(255)	NO	MUL	NULL	
food_type	varchar(100)	NO		NULL	
quantity	decimal(10,2)	NO		NULL	
preparation_time	datetime	NO		NULL	
expiry_time	datetime	NO		NULL	
food_condition	enum('FRESH','PACKAGED','PERISHABLE_FAST','COOKED')	NO		NULL	
location	text	NO		NULL	
status	varchar(50)	YES	MUL	AVAILABLE	
created_at	timestamp	YES		CURRENT_TIMESTAMP	DEFAULT_GENERATED

```
11 rows in set (0.00 sec)
```

-- 6. Create Views

```
mysql> CREATE OR REPLACE VIEW vw_available_donations AS
-> SELECT
->     fd.donation_id, fd.food_name, fd.food_type, fd.quantity, fd.food_condition, fd.expiry_time, fd.location,
->     donor.name AS donor_name, donor.phone AS donor_phone
-> FROM food_donation fd
-> JOIN food_donor donor ON fd.donor_id = donor.donor_id
-> WHERE fd.status = 'AVAILABLE' AND fd.expiry_time > NOW();
Query OK, 0 rows affected (0.03 sec)
```

```
mysql> describe vw_available_donations;
```

Field	Type	Null	Key	Default	Extra
donation_id	char(36)	NO		uuid()	DEFAULT_GENERATED
food_name	varchar(255)	NO		NULL	
food_type	varchar(100)	NO		NULL	
quantity	decimal(10,2)	NO		NULL	
food_condition	enum('FRESH','PACKAGED','PERISHABLE_FAST','COOKED')	NO		NULL	
expiry_time	datetime	NO		NULL	
location	text	NO		NULL	
donor_name	varchar(255)	NO		NULL	
donor_phone	varchar(20)	NO		NULL	

```
9 rows in set (0.01 sec)
```

```
mysql>
mysql> CREATE OR REPLACE VIEW vw_active_deliveries AS
-> SELECT
->     pd.delivery_id, fd.food_name, fd.quantity, c.name AS recipient_charity, c.address AS delivery_address,
->     v.name AS volunteer_name, v.phone AS volunteer_phone, pd.status AS delivery_status, pd.pickup_time
-> FROM pickup_delivery pd
-> JOIN food_donation fd ON pd.donation_id = fd.donation_id
-> JOIN charity_or_ngo c ON pd.charity_id = c.charity_id
-> LEFT JOIN volunteer v ON pd.volunteer_id = v.volunteer_id
-> WHERE pd.status IN ('ASSIGNED', 'PICKED_UP', 'IN_TRANSIT');
Query OK, 0 rows affected (0.05 sec)
```

```
mysql> describe vw_active_deliveries;
```

Field	Type	Null	Key	Default	Extra
delivery_id	char(36)	NO		uuid()	DEFAULT_GENERATED
food_name	varchar(255)	NO		NULL	
quantity	decimal(10,2)	NO		NULL	
recipient_charity	varchar(255)	NO		NULL	
delivery_address	text	NO		NULL	
volunteer_name	varchar(255)	YES		NULL	
volunteer_phone	varchar(30)	YES		NULL	
delivery_status	enum('ASSIGNED', 'PICKED_UP', 'IN_TRANSIT', 'DELIVERED', 'CANCELLED')	YES		ASSIGNED	
pickup_time	datetime	YES		NULL	

9 rows in set (0.01 sec)

Tables after

```
mysql> show tables
-> ;
```

Tables_in_ecofoodsync_db
allocation
charity_or_ngo
food_donation
food_donor
food_request
pickup_delivery
volunteer
vw_active_deliveries
vw_available_donations

9 rows in set (0.05 sec)

DML IMPLEMENTATION

6.1 INSERT Operations

- Insert records into `food_donor`
- Insert records into `charity_or_ngo`
- Insert records into `volunteer`
- Insert records into `food_donation`
- Insert records into `allocation`
- Insert records into `pickup_delivery`
- Insert records into `food_request`

6.1.1 Insert data into `food_donor`

```
mysql> INSERT INTO food_donor
-> (name, donor_type, phone, email, address)
-> VALUES
-> ('Green Leaf Restaurant', 'RESTAURANT', '9876543210',
-> 'greenleaf@gmail.com', 'Bangalore'),
->
-> ('FreshMart Supermarket', 'SUPERMARKET', '9876543211',
-> 'freshmart@gmail.com', 'Mysore'),
->
-> ('Annapurna Caterers', 'CATERER', '9876543212',
-> 'annapurna@gmail.com', 'Bangalore'),
->
-> ('Daily Meals Restaurant', 'RESTAURANT', '9876543213',
-> 'dailymeals@gmail.com', 'Chennai');
Query OK, 4 rows affected (0.19 sec)
Records: 4 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM food_donor;
+-----+-----+-----+-----+-----+-----+-----+
| donor_id | name | donor_type | phone | email | address | created_at |
+-----+-----+-----+-----+-----+-----+-----+
| ae67eb2f-bafe-11f1-bea9-842afdc08d7 | Green Leaf Restaurant | RESTAURANT | 9876543210 | greenleaf@gmail.com | Bangalore | 2026-09-28 11:07:23 |
| ae6a6b84-bafe-11f1-bea9-842afdc08d7 | FreshMart Supermarket | SUPERMARKET | 9876543211 | freshmart@gmail.com | Mysore | 2026-09-28 11:07:23 |
| ae6a7b9d-bafe-11f1-bea9-842afdc08d7 | Annapurna Caterers | CATERER | 9876543212 | annapurna@gmail.com | Bangalore | 2026-09-28 11:07:23 |
| ae6a7cca-bafe-11f1-bea9-842afdc08d7 | Daily Meals Restaurant | RESTAURANT | 9876543213 | dailymeals@gmail.com | Chennai | 2026-09-28 11:07:23 |
+-----+-----+-----+-----+-----+-----+-----+
4 rows in set (0.03 sec)
```

6.1.2 Insert data into charity_or_ngo

```
mysql> INSERT INTO charity_or_ngo
-> (name, contact_person, phone, email, address, operational_area, capacity)
-> VALUES
-> ('Helping Hands NGO', 'Anita Sharma', '9000000001',
-> 'helpinghands@gmail.com', 'Bangalore',
-> 'Bangalore Urban', 500),
->
-> ('Food For All Foundation', 'Ravi Kumar', '9000000002',
-> 'foodforall@gmail.com', 'Mysore',
-> 'Mysore', 300),
->
-> ('Hope Community Centre', 'Priya Rao', '9000000003',
-> 'hopecentre@gmail.com', 'Chennai',
-> 'Chennai', 400);
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM charity_or_ngo;
+-----+-----+-----+-----+-----+-----+-----+-----+
| charity_id | name | contact_person | phone | email | address | operational_area | capacity | created_at |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 5fffd61d7-baff-11f1-bea9-842afdc08d7 | Helping Hands NGO | Anita Sharma | 9000000001 | helpinghands@gmail.com | Bangalore | Bangalore Urban | 500 | 2026-09-28 11:12:21 |
| 5ffde41b-baff-11f1-bea9-842afdc08d7 | Food For All Foundation | Ravi Kumar | 9000000002 | foodforall@gmail.com | Mysore | Mysore | 300 | 2026-09-28 11:12:21 |
| 5ffde6ce-baff-11f1-bea9-842afdc08d7 | Hope Community Centre | Priya Rao | 9000000003 | hopecentre@gmail.com | Chennai | Chennai | 400 | 2026-09-28 11:12:21 |
+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.01 sec)

mysql>
```

6.1.3 Insert data into volunteer

```
mysql> INSERT INTO volunteer
-> (name, phone, email, availability_status, current_location)
-> VALUES
-> ('Arjun Singh', '9111111111',
-> 'arjun@gmail.com', TRUE, 'Bangalore'),
->
-> ('Sneha Rao', '9222222222',
-> 'sneha@gmail.com', TRUE, 'Mysore'),
->
-> ('Kiran Kumar', '9333333333',
-> 'kiran@gmail.com', TRUE, 'Chennai');
Query OK, 3 rows affected (0.03 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM volunteer;
```

volunteer_id	name	phone	email	availability_status	current_location	created_at
e68fff1e-baff-11f1-bea9-842afdc08d7	Arjun Singh	9111111111	arjun@gmail.com	1	Bangalore	2026-09-28 11:16:07
e6903241-baff-11f1-bea9-842afdc08d7	Sneha Rao	9222222222	sneha@gmail.com	1	Mysore	2026-09-28 11:16:07
e69033c6-baff-11f1-bea9-842afdc08d7	Kiran Kumar	9333333333	kiran@gmail.com	1	Chennai	2026-09-28 11:16:07

```
3 rows in set (0.00 sec)
```

6.1.4 Insert data into food_donation

```
mysql> INSERT INTO food_donation
-> (donor_id, food_name, food_type, quantity,
-> preparation_time, expiry_time, food_condition,
-> location, status)
-> VALUES
->
-> (
-> (SELECT donor_id
-> FROM food_donor
-> WHERE email = 'greenleaf@gmail.com'),
->
-> 'Vegetable Biryani',
-> 'Cooked',
-> 50.00,
-> '2026-09-28 09:00:00',
-> '2026-09-28 18:00:00',
-> 'COOKED',
-> 'Bangalore',
-> 'AVAILABLE'
-> ),
->
-> (
-> (SELECT donor_id
-> FROM food_donor
-> WHERE email = 'freshmart@gmail.com'),
->
-> 'Fresh Bread',
-> 'Bakery',
-> 30.00,
-> '2026-09-28 08:00:00',
-> '2026-09-29 12:00:00',
-> 'FRESH',
-> 'Mysore',
-> 'AVAILABLE'
-> ),
->
-> (
-> (SELECT donor_id
-> FROM food_donor
-> WHERE email = 'annapurna@gmail.com'),
->
```

```

--> 'Rice and Curry',
--> 'Cooked',
--> 75.00,
--> '2026-09-28 10:00:00',
--> '2026-09-28 20:00:00',
--> 'COOKED',
--> 'Bangalore',
--> 'AVAILABLE'
--> );
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0

mysql> SELECT * FROM food_donation;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| donation_id | donor_id | food_name | food_type | quantity | preparation_time | expiry_time |
+-----+-----+-----+-----+-----+-----+-----+
| 0a391799-bb00-11f1-bea9-842afdc08d7 | ae67eb2f-bafe-11f1-bea9-842afdc08d7 | Vegetable Biryani | Cooked | 50.00 | 2026-09-28 09:00:00 | 2026-09-28 18:00:00 |
| 0a3951a7-bb00-11f1-bea9-842afdc08d7 | ae6a6b84-bafe-11f1-bea9-842afdc08d7 | Fresh Bread | Bakery | 30.00 | 2026-09-28 08:00:00 | 2026-09-29 12:00:00 |
| 0a3968b6-bb00-11f1-bea9-842afdc08d7 | ae6a7b9d-bafe-11f1-bea9-842afdc08d7 | Rice and Curry | Cooked | 75.00 | 2026-09-28 10:00:00 | 2026-09-28 20:00:00 |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

```

6.1.5 Insert data into allocation

```

mysql> INSERT INTO allocation
--> (donation_id, charity_id, allocated_quantity, status)
--> VALUES
--> (
--> (SELECT donation_id
--> FROM food_donation
--> WHERE food_name = 'Vegetable Biryani'),
--> (SELECT charity_id
--> FROM charity_or_ngo
--> WHERE email = 'helpinghands@gmail.com'),
--> 30.00,
--> 'APPROVED'
--> ),
--> (
--> (SELECT donation_id
--> FROM food_donation
--> WHERE food_name = 'Fresh Bread'),
--> (SELECT charity_id
--> FROM charity_or_ngo
--> WHERE email = 'foodforall@gmail.com'),
--> 20.00,
--> 'PENDING'
--> );
Query OK, 2 rows affected (0.06 sec)

mysql> SELECT * FROM allocation;
+-----+-----+-----+-----+-----+-----+
| allocation_id | donation_id | charity_id | allocated_quantity | allocation_time | status |
+-----+-----+-----+-----+-----+-----+
| 6347875b-bb00-11f1-bea9-842afdc08d7 | 0a391799-bb00-11f1-bea9-842afdc08d7 | 5ffd61d7-baff-11f1-bea9-842afdc08d7 | 30.00 | 2026-09-28 11:19:36 | APPROV |
| 63480228-bb00-11f1-bea9-842afdc08d7 | 0a3951a7-bb00-11f1-bea9-842afdc08d7 | 5ffde41b-baff-11f1-bea9-842afdc08d7 | 20.00 | 2026-09-28 11:19:36 | PENDIN |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)

```

6.1.6 Insert data into pickup_delivery

```
mysql> INSERT INTO pickup_delivery
-> (donation_id, volunteer_id, charity_id,
-> pickup_time, delivery_time, vehicle_type, status, delivery_notes)
-> VALUES
->
-> (
-> (SELECT donation_id
-> FROM food_donation
-> WHERE food_name = 'Vegetable Biryani'),
->
-> (SELECT volunteer_id
-> FROM volunteer
-> WHERE email = 'arjun@gmail.com'),
->
-> (SELECT charity_id
-> FROM charity_or_ngo
-> WHERE email = 'helpinghands@gmail.com'),
->
-> '2026-09-28 12:00:00',
-> '2026-09-28 13:00:00',
-> 'Van',
```

```

-> WHERE food_name = 'Vegetable Biryani'),
->
-> (SELECT volunteer_id
-> FROM volunteer
-> WHERE email = 'arjun@gmail.com'),
->
-> (SELECT charity_id
-> FROM charity_or_ngo
-> WHERE email = 'helpinghands@gmail.com'),
->
-> '2026-09-28 12:00:00',
-> '2026-09-28 13:00:00',
-> 'Van',
-> 'DELIVERED',
-> 'Food delivered successfully'
-> ),
->
-> (
-> (SELECT donation_id
-> FROM food_donation
-> WHERE food_name = 'Fresh Bread'),
->
-> (SELECT volunteer_id
-> FROM volunteer
-> WHERE email = 'sneha@gmail.com'),
->
-> (SELECT charity_id
-> FROM charity_or_ngo
-> WHERE email = 'foodforall@gmail.com'),
->
-> '2026-09-28 11:00:00',
-> NULL,
-> 'Mini Truck',
-> 'IN_TRANSIT',
-> 'Delivery in progress'
-> );
Query OK, 2 rows affected (0.03 sec)
Records: 2 Duplicates: 0 Warnings: 0

```

```
mysql> SELECT * FROM pickup_delivery;
```

delivery_id	delivery_time	donation_id	vehicle_type	status	delivery_notes	volunteer_id	charity_id	pickup_time
8f14e9c8-bb00-11f1-bea9-842afdc008d7	2026-09-28 12:00:00	0a391799-bb00-11f1-bea9-842afdc008d7	Van	DELIVERED	Food delivered successfully	e68ffff1e-baff-11f1-bea9-842afdc008d7	5ffd61d7-baff-11f1-bea9-842afdc008d7	2026-09-28
8f156768-bb00-11f1-bea9-842afdc008d7	2026-09-28 11:00:00	0a3951a7-bb00-11f1-bea9-842afdc008d7	Mini Truck	IN_TRANSIT	Delivery in progress	e6903241-baff-11f1-bea9-842afdc008d7	5ffd641b-baff-11f1-bea9-842afdc008d7	2026-09-28

6.1.7 Insert data into food_request

```
mysql> INSERT INTO food_request
-> (charity_id, required_food_type, required_quantity,
-> urgency_level, status)
-> VALUES
->
-> (
-> (SELECT charity_id
-> FROM charity_or_ngo
-> WHERE email = 'helpinghands@gmail.com'),
->
-> 'Cooked Food',
-> 100.00,
-> 'HIGH',
-> 'OPEN'
-> ),
->
-> (
-> (SELECT charity_id
-> FROM charity_or_ngo
-> WHERE email = 'foodforall@gmail.com'),
->
-> WHERE email = 'helpinghands@gmail.com'),
->
-> 'Cooked Food',
-> 100.00,
-> 'HIGH',
-> 'OPEN'
-> ),
->
-> (
-> (SELECT charity_id
-> FROM charity_or_ngo
-> WHERE email = 'foodforall@gmail.com'),
->
-> 'Bakery',
-> 50.00,
-> 'MEDIUM',
-> 'OPEN'
-> ),
->
-> (
-> (SELECT charity_id
-> FROM charity_or_ngo
-> WHERE email = 'hopecentre@gmail.com'),
->
-> 'Cooked Food',
-> 80.00,
-> 'CRITICAL',
-> 'OPEN'
-> );
Query OK, 3 rows affected (0.02 sec)
Records: 3 Duplicates: 0 Warnings: 0
```

```
mysql> SELECT * FROM food_request;
```

request_id	charity_id	required_food_type	required_quantity	urgency_level	request_time	status
e31cedad-bb00-11f1-bea9-842afdc08d7	5ffd61d7-baff-11f1-bea9-842afdc08d7	Cooked Food	100.00	HIGH	2026-09-28 11:23:10	OPEN
e31d2ee1-bb00-11f1-bea9-842afdc08d7	5ffde41b-baff-11f1-bea9-842afdc08d7	Bakery	50.00	MEDIUM	2026-09-28 11:23:10	OPEN
e31d3338-bb00-11f1-bea9-842afdc08d7	5ffde6ce-baff-11f1-bea9-842afdc08d7	Cooked Food	80.00	CRITICAL	2026-09-28 11:23:10	OPEN

3 rows in set (0.00 sec)

6.2 UPDATE Operations

6.2.1 Update Food Donation Status

```
mysql> UPDATE food_donation
  -> SET status = 'CLAIMED'
  -> WHERE food_name = 'Vegetable Biryani';
Query OK, 1 row affected (0.04 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> SELECT food_name, status
  -> FROM food_donation
  -> WHERE food_name = 'Vegetable Biryani';
+-----+-----+
| food_name      | status |
+-----+-----+
| Vegetable Biryani | CLAIMED |
+-----+-----+
1 row in set (0.00 sec)
```

6.2.2 Update Volunteer Availability

```
mysql> UPDATE volunteer
  -> SET availability_status = FALSE,
  ->     current_location = 'Bangalore'
  -> WHERE email = 'arjun@gmail.com';
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT name, availability_status, current_location
  -> FROM volunteer
  -> WHERE email = 'arjun@gmail.com';
+-----+-----+-----+
| name      | availability_status | current_location |
+-----+-----+-----+
| Arjun Singh | 0 | Bangalore |
+-----+-----+-----+
1 row in set (0.00 sec)
```

6.2.3 Update Charity Capacity

```
mysql> UPDATE charity_or_ngo
  -> SET capacity = capacity + 100
  -> WHERE email = 'helpinghands@gmail.com';
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT name, capacity
  -> FROM charity_or_ngo
  -> WHERE email = 'helpinghands@gmail.com';
+-----+-----+
| name          | capacity |
+-----+-----+
| Helping Hands NGO | 600 |
+-----+-----+
1 row in set (0.00 sec)
```

6.2.4 Update Allocation Status

```
mysql> UPDATE allocation
-> SET status = 'APPROVED'
-> WHERE donation_id = (
->     SELECT donation_id
->     FROM food_donation
->     WHERE food_name = 'Fresh Bread'
-> );
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT allocation_id, allocated_quantity, status
-> FROM allocation
-> WHERE donation_id = (
->     SELECT donation_id
->     FROM food_donation
->     WHERE food_name = 'Fresh Bread'
-> );
```

allocation_id	allocated_quantity	status
63480228-bb00-11f1-bea9-842afdc08d7	20.00	APPROVED

1 row in set (0.01 sec)

6.3 DELETE Operations

6.3.1 Delete a Food Request

```
mysql> DELETE FROM food_request
-> WHERE charity_id = (
->     SELECT charity_id
->     FROM charity_or_ngo
->     WHERE email = 'foodforall@gmail.com'
-> )
-> AND urgency_level = 'MEDIUM';
Query OK, 1 row affected (0.02 sec)

mysql> SELECT *
-> FROM food_request;
```

request_id	charity_id	required_food_type	required_quantity	urgency_level	request_time	status
e31cedad-bb00-11f1-bea9-842afdc08d7	5ffd61d7-baff-11f1-bea9-842afdc08d7	Cooked Food	100.00	HIGH	2026-09-28 11:23:10	OPEN
e31d3338-bb00-11f1-bea9-842afdc08d7	5ffde6ce-baff-11f1-bea9-842afdc08d7	Cooked Food	80.00	CRITICAL	2026-09-28 11:23:10	OPEN

2 rows in set (0.00 sec)

6.3.2 Delete an Unused Volunteer

```
mysql> DELETE FROM volunteer
-> WHERE email = 'kiran@gmail.com';
Query OK, 1 row affected (0.09 sec)

mysql> SELECT *
-> FROM volunteer;
```

volunteer_id	name	phone	email	availability_status	current_location	created_at
e68ff1e-baff-11f1-bea9-842afdc08d7	Arjun Singh	9111111111	arjun@gmail.com	0	Bangalore	2026-09-28 11:16:07
e6903241-baff-11f1-bea9-842afdc08d7	Sneha Rao	9222222222	sneha@gmail.com	1	Mysore	2026-09-28 11:16:07

2 rows in set (0.01 sec)

6.4 SELECT Operations

6.4.1 Display All Food Donations

```
mysql> SELECT *
-> FROM food_donation;
```

donation_id	location	status	donor_id	created_at	food_name	food_type	quantity	preparation_time	expiry_time
0a391799-bb00-11f1-bea9-842afdc08d7	Bangalore	CLAIMED	ae67eb2f-bafe-11f1-bea9-842afdc08d7	2026-09-28 11:17:07	Vegetable Biryani	Cooked	50.00	2026-09-28 09:00:00	2026-09-28 18:00:00
0a3951a7-bb00-11f1-bea9-842afdc08d7	Mysore	AVAILABLE	ae6a6b84-bafe-11f1-bea9-842afdc08d7	2026-09-28 11:17:07	Fresh Bread	Bakery	30.00	2026-09-28 08:00:00	2026-09-29 12:00:00
0a3968b6-bb00-11f1-bea9-842afdc08d7	Bangalore	AVAILABLE	ae6a7b9d-bafe-11f1-bea9-842afdc08d7	2026-09-28 11:17:07	Rice and Curry	Cooked	75.00	2026-09-28 10:00:00	2026-09-28 20:00:00

6.4.2 Display Available Food

```
mysql> SELECT food_name, food_type, quantity, location, expiry_time
-> FROM food_donation
-> WHERE status = 'AVAILABLE';
```

food_name	food_type	quantity	location	expiry_time
Fresh Bread	Bakery	30.00	Mysore	2026-09-29 12:00:00
Rice and Curry	Cooked	75.00	Bangalore	2026-09-28 20:00:00

2 rows in set (0.01 sec)

6.4.3 Display High-Quantity Donations

```
mysql> SELECT food_name, quantity, location
-> FROM food_donation
-> WHERE quantity > 40;
```

food_name	quantity	location
Vegetable Biryani	50.00	Bangalore
Rice and Curry	75.00	Bangalore

2 rows in set (0.01 sec)

6.4.4 Display Donations by Earliest Expiry

```
mysql> SELECT food_name, quantity, expiry_time
-> FROM food_donation
-> ORDER BY expiry_time ASC;
```

food_name	quantity	expiry_time
Vegetable Biryani	50.00	2026-09-28 18:00:00
Rice and Curry	75.00	2026-09-28 20:00:00
Fresh Bread	30.00	2026-09-29 12:00:00

3 rows in set (0.01 sec)

6.4.5 Display High-Urgency Food Requests

```
mysql> SELECT required_food_type,
->         required_quantity,
->         urgency_level,
->         status
-> FROM food_request
-> WHERE urgency_level IN ('HIGH', 'CRITICAL');
```

required_food_type	required_quantity	urgency_level	status
Cooked Food	100.00	HIGH	OPEN
Cooked Food	80.00	CRITICAL	OPEN

2 rows in set (0.01 sec)

6.5 JOIN Operations

6.5.1 Donor + Food Donation

Question: Display the donor name, food name, quantity and location.

```
mysql> SELECT
->     d.name AS donor_name,
->     fd.food_name,
->     fd.quantity,
->     fd.location
-> FROM food_donor d
-> JOIN food_donation fd
->     ON d.donor_id = fd.donor_id;
```

donor_name	food_name	quantity	location
Green Leaf Restaurant	Vegetable Biryani	50.00	Bangalore
FreshMart Supermarket	Fresh Bread	30.00	Mysore
Annapurna Caterers	Rice and Curry	75.00	Bangalore

3 rows in set (0.02 sec)

6.5.2 Donation + Charity

Question: Display which food was allocated to which charity.

```
mysql> SELECT
->     fd.food_name,
->     c.name AS charity_name,
->     a.allocated_quantity,
->     a.status
-> FROM allocation a
-> JOIN food_donation fd
->     ON a.donation_id = fd.donation_id
-> JOIN charity_or_ngo c
->     ON a.charity_id = c.charity_id;
```

food_name	charity_name	allocated_quantity	status
Vegetable Biryani	Helping Hands NGO	30.00	APPROVED
Fresh Bread	Food For All Foundation	20.00	APPROVED

2 rows in set (0.01 sec)

6.5.3 Complete Delivery Information ★

Question: Display the food, donor, volunteer and charity involved in each delivery.

```
mysql> SELECT
->     fd.food_name,
->     d.name AS donor_name,
->     v.name AS volunteer_name,
->     c.name AS charity_name,
->     pd.status
-> FROM pickup_delivery pd
-> JOIN food_donation fd
->     ON pd.donation_id = fd.donation_id
-> JOIN food_donor d
->     ON fd.donor_id = d.donor_id
-> LEFT JOIN volunteer v
->     ON pd.volunteer_id = v.volunteer_id
-> JOIN charity_or_ngo c
->     ON pd.charity_id = c.charity_id;
```

food_name	donor_name	volunteer_name	charity_name	status
Vegetable Biryani	Green Leaf Restaurant	Anjun Singh	Helping Hands NGO	DELIVERED
Fresh Bread	FreshMart Supermarket	Sneha Rao	Food For All Foundation	IN_TRANSIT

2 rows in set (0.02 sec)

6.6 Aggregate Functions

6.6.1 COUNT()

Question:

Find the total number of food donations.

```
mysql> SELECT COUNT(*) AS total_donations
-> FROM food_donation;
+-----+
| total_donations |
+-----+
|                3 |
+-----+
1 row in set (0.08 sec)
```

6.6.2 SUM()

Question:

Find the total quantity of food donated.

```
mysql> SELECT SUM(quantity) AS total_food_quantity
-> FROM food_donation;
+-----+
| total_food_quantity |
+-----+
|                155.00 |
+-----+
1 row in set (0.02 sec)
```

6.6.3 AVG()

Question:

Find the average quantity of food donated.

```
mysql> SELECT AVG(quantity) AS average_food_quantity
-> FROM food_donation;
+-----+
| average_food_quantity |
+-----+
|                51.666667 |
+-----+
1 row in set (0.00 sec)
```

6.6.4 MAX()

Question:

Find the largest food donation.

```
mysql> SELECT MAX(quantity) AS maximum_donation
-> FROM food_donation;
+-----+
| maximum_donation |
+-----+
|          75.00 |
+-----+
1 row in set (0.00 sec)
```

6.6.5 MIN()

Question:

Find the smallest food donation.

```
mysql> SELECT MIN(quantity) AS minimum_donation
-> FROM food_donation;
+-----+
| minimum_donation |
+-----+
|          30.00 |
+-----+
1 row in set (0.00 sec)
```

6.6.6 All Aggregate Functions Together

```
mysql> SELECT
-> COUNT(*) AS total_donations,
-> SUM(quantity) AS total_quantity,
-> AVG(quantity) AS average_quantity,
-> MAX(quantity) AS maximum_quantity,
-> MIN(quantity) AS minimum_quantity
-> FROM food_donation;
+-----+-----+-----+-----+-----+
| total_donations | total_quantity | average_quantity | maximum_quantity | minimum_quantity |
+-----+-----+-----+-----+-----+
|          3 |          155.00 |          51.666667 |          75.00 |          30.00 |
+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

6.7 GROUP BY Operations

6.7.1 Total Food Donated by Each Donor

```
mysql> SELECT
->     d.name AS donor_name,
->     SUM(fd.quantity) AS total_quantity
-> FROM food_donor d
-> JOIN food_donation fd
->     ON d.donor_id = fd.donor_id
-> GROUP BY d.donor_id, d.name;
```

donor_name	total_quantity
Green Leaf Restaurant	50.00
FreshMart Supermarket	30.00
Annapurna Caterers	75.00

3 rows in set (0.04 sec)

6.7.2 Total Food Received by Each Charity

```
mysql> SELECT
->     c.name AS charity_name,
->     SUM(a.allocated_quantity) AS total_received
-> FROM charity_or_ngo c
-> JOIN allocation a
->     ON c.charity_id = a.charity_id
-> GROUP BY c.charity_id, c.name;
```

charity_name	total_received
Helping Hands NGO	30.00
Food For All Foundation	20.00

2 rows in set (0.01 sec)

6.8 HAVING Operations

6.8.1 Donors with More Than 40 kg Total Donations

```
mysql> SELECT
->     d.name AS donor_name,
->     SUM(fd.quantity) AS total_quantity
-> FROM food_donor d
-> JOIN food_donation fd
->     ON d.donor_id = fd.donor_id
-> GROUP BY d.donor_id, d.name
-> HAVING SUM(fd.quantity) > 40;
```

donor_name	total_quantity
Green Leaf Restaurant	50.00
Annapurna Caterers	75.00

2 rows in set (0.01 sec)

6.8.2 Charities Receiving More Than 20 kg

```
mysql> SELECT
->     c.name AS charity_name,
->     SUM(a.allocated_quantity) AS total_received
-> FROM charity_or_ngo c
-> JOIN allocation a
->     ON c.charity_id = a.charity_id
-> GROUP BY c.charity_id, c.name
-> HAVING SUM(a.allocated_quantity) > 20;
```

charity_name	total_received
Helping Hands NGO	30.00

1 row in set (0.00 sec)

6.9 ORDER BY Operations

6.9.1 Donations by Quantity

```
mysql> SELECT
->     food_name,
->     quantity,
->     location
-> FROM food_donation
-> ORDER BY quantity DESC;
```

food_name	quantity	location
Rice and Curry	75.00	Bangalore
Vegetable Biryani	50.00	Bangalore
Fresh Bread	30.00	Mysore

3 rows in set (0.01 sec)

6.9.2 Donations by Earliest Expiry

```
mysql> SELECT
->     food_name,
->     quantity,
->     expiry_time
-> FROM food_donation
-> ORDER BY expiry_time ASC;
```

food_name	quantity	expiry_time
Vegetable Biryani	50.00	2026-09-28 18:00:00
Rice and Curry	75.00	2026-09-28 20:00:00
Fresh Bread	30.00	2026-09-29 12:00:00

3 rows in set (0.00 sec)

6.10 Subqueries

6.10.1 Donations Above Average Quantity

```
mysql> SELECT
->     food_name,
->     quantity
-> FROM food_donation
-> WHERE quantity >
-> (
->     SELECT AVG(quantity)
->     FROM food_donation
-> );
```

food_name	quantity
Rice and Curry	75.00

1 row in set (0.01 sec)

6.10.2 Donation with Maximum Quantity

```
mysql> SELECT
->     food_name,
->     quantity
-> FROM food_donation
-> WHERE quantity =
-> (
->     SELECT MAX(quantity)
->     FROM food_donation
-> );
```

food_name	quantity
Rice and Curry	75.00

1 row in set (0.00 sec)

6.10.3 Charities Above Average Capacity

```
mysql> SELECT
->     name,
->     capacity
-> FROM charity_or_ngo
-> WHERE capacity >
-> (
->     SELECT AVG(capacity)
->     FROM charity_or_ngo
-> );
```

name	capacity
Helping Hands NGO	600

1 row in set (0.01 sec)

6.11 Complex Queries

6.11.1 Donors with More Than 40 kg, Sorted by Quantity

```
mysql> SELECT
->     d.name AS donor_name,
->     SUM(fd.quantity) AS total_quantity
-> FROM food_donor d
-> JOIN food_donation fd
->     ON d.donor_id = fd.donor_id
-> GROUP BY d.donor_id, d.name
-> HAVING SUM(fd.quantity) > 40
-> ORDER BY total_quantity DESC;
```

```
+-----+-----+
| donor_name | total_quantity |
+-----+-----+
| Annapurna Caterers | 75.00 |
| Green Leaf Restaurant | 50.00 |
+-----+-----+
2 rows in set (0.00 sec)
```

6.11.2 Complete Food Allocation Information

Display:

- Food name
- Donor
- Charity
- Allocated quantity
- Allocation status

```
mysql> SELECT
->     fd.food_name,
->     d.name AS donor_name,
->     c.name AS charity_name,
->     a.allocated_quantity,
->     a.status
-> FROM allocation a
-> JOIN food_donation fd
->     ON a.donation_id = fd.donation_id
-> JOIN food_donor d
->     ON fd.donor_id = d.donor_id
-> JOIN charity_or_ngo c
->     ON a.charity_id = c.charity_id
-> ORDER BY a.allocated_quantity DESC;
```

```
+-----+-----+-----+-----+-----+
| food_name | donor_name | charity_name | allocated_quantity | status |
+-----+-----+-----+-----+-----+
| Vegetable Biryani | Green Leaf Restaurant | Helping Hands NGO | 30.00 | APPROVED |
| Fresh Bread | FreshMart Supermarket | Food For All Foundation | 20.00 | APPROVED |
+-----+-----+-----+-----+-----+
2 rows in set (0.01 sec)
```

ADVANCED DATABASE MANAGEMENT SYSTEM

6. VIEW

6.1 Creation of View for Active Deliveries

```
PES2UG24CS660_PES2UG24CS667>CREATE OR REPLACE VIEW vw_active_deliveries AS
-> SELECT
->   pd.delivery_id,
->   fd.food_name,
->   fd.quantity,
->   c.name AS recipient_charity,
->   c.location AS delivery_address,
->   v.name AS volunteer_name,
->   v.phone AS volunteer_phone,
->   pd.status AS delivery_status,
->   pd.pickup_time
-> FROM pickup_delivery pd
-> JOIN food_donation fd
->   ON pd.donation_id = fd.donation_id
-> JOIN charity_or_ngo c
->   ON pd.charity_id = c.charity_id
-> JOIN volunteer v
->   ON pd.volunteer_id = v.volunteer_id;
Query OK, 0 rows affected (0.05 sec)

PES2UG24CS660_PES2UG24CS667>SELECT * FROM vw_active_deliveries;
```

delivery_id	food_name	quantity	recipient_charity	delivery_address	volunteer_name	volunteer_phone	delivery_status	pickup_time
cea46fe4-bf9f-11f1-891a-0a0027000002	Vegetable Biryani	50.00	Helping Hands NGO	Bangalore	Arjun Singh	9111111111	DELIVERED	2026-09-28 12:00:00
cea48576-bf9f-11f1-891a-0a0027000002	Fresh Bread	30.00	Food For All Foundation	Mysore	Sneha Rao	9222222222	IN_TRANSIT	2026-09-28 11:00:00

```
2 rows in set (0.00 sec)

PES2UG24CS660_PES2UG24CS667>
```

7. INDEXING AND QUERY OPTIMIZATION

7.1 Displaying Indexes on food_donation

```
PES2UG24CS660_PES2UG24CS667>SHOW INDEX FROM food_donation;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
food_donation	0	PRIMARY	1	donation_id	A	3	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_food_donation_donor	1	donor_id	A	3	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_donation_status_expiry	1	status	A	1	NULL	NULL	YES	BTREE			YES	NULL
food_donation	1	idx_donation_status_expiry	2	expiry_time	A	3	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_donation_status_expiry	3	food_condition	A	3	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_food_name_fulltext	1	food_name	NULL	3	NULL	NULL		FULLTEXT			YES	NULL

```
6 rows in set (0.00 sec)

PES2UG24CS660_PES2UG24CS667>
```

```
PES2UG24CS660_PES2UG24CS667>EXPLAIN
-> SELECT *
-> FROM food_donation
-> WHERE donor_id = (
->   SELECT donor_id
->   FROM food_donor
->   WHERE email = 'greenleaf@gmail.com'
-> );
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	PRIMARY	food_donation	NULL	ref	idx_food_donation_donor	idx_food_donation_donor	144	const	1	100.00	Using where
2	SUBQUERY	food_donor	NULL	const	email	email	1022	const	1	100.00	Using index

```
2 rows in set, 1 warning (0.00 sec)

PES2UG24CS660_PES2UG24CS667>
```

7.2 Composite Index Optimization

```
PES2UG24CS660_PES2UG24CS667>EXPLAIN
-> SELECT *
-> FROM food_donation
-> WHERE status = 'AVAILABLE'
-> ORDER BY expiry_time;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	food_donation	NULL	ref	idx_donation_status_expiry	idx_donation_status_expiry	203	const	3	100.00	NULL

```
1 row in set, 1 warning (0.00 sec)

PES2UG24CS660_PES2UG24CS667>
```

8. STORED PROCEDURES

8.1 Stored Procedure to Retrieve Available Food by Location

```
PES2UG24CS660_PES2UG24CS667>DELIMITER ;
PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>CREATE PROCEDURE GetAvailableFoodByLocation(
->    IN p_location VARCHAR(100)
-> )
-> BEGIN
->     SELECT
->         donation_id,
->         food_name,
->         food_type,
->         quantity,
->         food_condition,
->         expiry_time,
->         location,
->         status
->     FROM food_donation
->     WHERE location = p_location
->         AND status = 'AVAILABLE'
->         AND expiry_time > NOW()
->     ORDER BY expiry_time;
-> END $$
ERROR 1304 (42000): PROCEDURE GetAvailableFoodByLocation already exists
PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>DELIMITER ;
PES2UG24CS660_PES2UG24CS667>
```

8.2 Stored Procedure for Donation Allocation summary

```
PES2UG24CS660_PES2UG24CS667>CREATE PROCEDURE GetDonationAllocationSummary()
-> BEGIN
->     SELECT
->         fd.donation_id,
->         fd.food_name,
->         fd.quantity AS donated_quantity,
->         COALESCE(SUM(a.allocated_quantity), 0) AS allocated_quantity,
->         fd.quantity - COALESCE(SUM(a.allocated_quantity), 0)
->             AS remaining_quantity
->     FROM food_donation fd
->     LEFT JOIN allocation a
->         ON fd.donation_id = a.donation_id
->     GROUP BY
->         fd.donation_id,
->         fd.food_name,
->         fd.quantity
->     ORDER BY fd.food_name;
-> END $$
ERROR 1304 (42000): PROCEDURE GetDonationAllocationSummary already exists
PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>DELIMITER ;
PES2UG24CS660_PES2UG24CS667>
```

```
PES2UG24CS660_PES2UG24CS667>CALL GetDonationAllocationSummary();
```

donation_id	food_name	donated_quantity	allocated_quantity	remaining_quantity
ce9dc57b-bf9f-11f1-891a-0a0027000002	Fresh Bread	30.00	20.00	10.00
ce9dca92-bf9f-11f1-891a-0a0027000002	Rice and Curry	75.00	0.00	75.00
ce9db9ad-bf9f-11f1-891a-0a0027000002	Vegetable Biryani	50.00	30.00	20.00

```
3 rows in set (0.00 sec)
```

```
Query OK, 0 rows affected (0.02 sec)
```

9. STORED FUNCTION

9.1 Creation of Stored Function

```

PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>CREATE FUNCTION GetRemainingDonationQuantity(
->    p_donation_id CHAR(36)
-> )
-> RETURNS DECIMAL(10,2)
-> DETERMINISTIC
-> READS SQL DATA
-> BEGIN
->     DECLARE donated_qty DECIMAL(10,2);
->     DECLARE allocated_qty DECIMAL(10,2);
->
->     SELECT quantity
->     INTO donated_qty
->     FROM food_donation
->     WHERE donation_id = p_donation_id;
->
->     SELECT COALESCE(SUM(allocated_quantity), 0)
->     INTO allocated_qty
->     FROM allocation
->     WHERE donation_id = p_donation_id;
->
->     RETURN donated_qty - allocated_qty;
-> END $$
ERROR 1304 (42000): FUNCTION GetRemainingDonationQuantity already exists
PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>DELIMITER ;
PES2UG24CS660_PES2UG24CS667>

```

9.2 Testing the Stored Function

```

PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>DELIMITER ;
PES2UG24CS660_PES2UG24CS667>SELECT
->    food_name,
->    quantity AS donated_quantity,
->    GetRemainingDonationQuantity(donation_id)
->    AS remaining_quantity
-> FROM food_donation;

```

food_name	donated_quantity	remaining_quantity
Vegetable Biryani	50.00	20.00
Fresh Bread	30.00	10.00
Rice and Curry	75.00	75.00

```

3 rows in set (0.00 sec)

PES2UG24CS660_PES2UG24CS667>

```

9.3 Verification of Stored Function

```

PES2UG24CS660_PES2UG24CS667>SHOW FUNCTION STATUS
-> WHERE Db = DATABASE();

```

Db	Name	Type	Definer	Modified	Created	Security_type	Comment	character_set_client	collation_connection	Database Collation
ecofoodsync_db	GetRemainingDonationQuantity	FUNCTION	root@localhost	2026-10-04 08:30:53	2026-10-04 08:30:53	DEFINER		cp850	cp850_general_ci	utf8mb4_0900_ai_ci

```

1 row in set (0.00 sec)

PES2UG24CS660_PES2UG24CS667>

```

10. TRIGGERS

10.1 Trigger to Prevent Excess Food Allocation

```

PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>CREATE TRIGGER before_allocation_insert
-> BEFORE INSERT ON allocation
-> FOR EACH ROW
-> BEGIN
->   DECLARE available_qty DECIMAL(10,2);
->
->   SELECT quantity - COALESCE(
->     (
->       SELECT SUM(allocated_quantity)
->       FROM allocation
->       WHERE donation_id = NEW.donation_id
->     ),
->     0
->   )
->   INTO available_qty
->   FROM food_donation
->   WHERE donation_id = NEW.donation_id;
->
->   IF NEW.allocated_quantity > available_qty THEN
->     SIGNAL SQLSTATE '45000'
->     SET MESSAGE_TEXT =
->     'Allocation exceeds available donation quantity';
->   END IF;
-> END $$
ERROR 1359 (HY000): Trigger already exists

```

10.2 Trigger to Validate Food Expiry Time

```

PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>CREATE TRIGGER before_food_donation_insert
-> BEFORE INSERT ON food_donation
-> FOR EACH ROW
-> BEGIN
->   IF NEW.expiry_time <= NEW.preparation_time THEN
->     SIGNAL SQLSTATE '45000'
->     SET MESSAGE_TEXT =
->     'Expiry time must be after preparation time';
->   END IF;
-> END $$
ERROR 1359 (HY000): Trigger already exists
PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>DELIMITER ;
PES2UG24CS660_PES2UG24CS667>START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

```

10.3 Testing the Allocation Trigger — Valid Operation

```

PES2UG24CS660_PES2UG24CS667>DELIMITER ;
PES2UG24CS660_PES2UG24CS667>START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>INSERT INTO allocation
-> (
->     donation_id,
->     charity_id,
->     allocated_quantity,
->     status
-> )
-> VALUES
-> (
->     (
->         SELECT donation_id
->         FROM food_donation
->         WHERE food_name = 'Rice and Curry'
->         LIMIT 1
->     ),
->     (
->         SELECT charity_id
->         FROM charity_or_ngo
->         WHERE name = 'Food For All Foundation'
->         LIMIT 1
->     ),
->     5.00,
->     'PENDING'
-> );
Query OK, 1 row affected (0.01 sec)

PES2UG24CS660_PES2UG24CS667>SELECT
->     fd.food_name,
->     a.allocated_quantity,
->     a.status
-> FROM allocation a
-> JOIN food_donation fd
->     ON a.donation_id = fd.donation_id
-> WHERE fd.food_name = 'Rice and Curry';
+-----+-----+-----+
| food_name | allocated_quantity | status |
+-----+-----+-----+
| Rice and Curry | 5.00 | PENDING |
+-----+-----+-----+
1 row in set (0.00 sec)

```

10.4 Testing the Allocation Trigger — Invalid Operation

```

PES2UG24CS660_PES2UG24CS667>INSERT INTO allocation
-> (
->     donation_id,
->     charity_id,
->     allocated_quantity,
->     status
-> )
-> VALUES
-> (
->     (
->         SELECT donation_id
->         FROM food_donation
->         WHERE food_name = 'Vegetable Biryani'
->         LIMIT 1
->     ),
->     (
->         SELECT charity_id
->         FROM charity_or_ngo
->         WHERE name = 'Helping Hands NGO'
->         LIMIT 1
->     ),
->     25.00,
->     'PENDING'
-> );
ERROR 1644 (45000): Allocation exceeds available donation quantity
PES2UG24CS660_PES2UG24CS667>

```

10.5 Testing the Food Expiry Trigger

```

ERROR 1644 (45000): Allocation exceeds available donation quantity
PES2UG24CS660_PES2UG24CS667>INSERT INTO food_donation
-> (
->     donor_id,
->     food_name,
->     food_type,
->     quantity,
->     preparation_time,
->     expiry_time,
->     food_condition,
->     location,
->     status
-> )
-> VALUES
-> (
->     (
->         SELECT donor_id
->         FROM food_donor
->         WHERE email = 'greenleaf@gmail.com'
->         LIMIT 1
->     ),
->     'Test Food',
->     'Cooked',
->     10.00,
->     '2026-10-04 15:00:00',
->     '2026-10-04 14:00:00',
->     'COOKED',
->     'Bangalore',
->     'AVAILABLE'
-> );
ERROR 1644 (45000): Expiry time must be after preparation time
PES2UG24CS660_PES2UG24CS667>|

```

10.6 Verifying That the Invalid Donation Was Not Inserted

```

PES2UG24CS660_PES2UG24CS667>SELECT *
-> FROM food_donation
-> WHERE food_name = 'Test Food';
Empty set (0.00 sec)

PES2UG24CS660_PES2UG24CS667>|

```

11. TRANSACTIONS

- START TRANSACTION
- SAVEPOINT

```

PES2UG24CS660_PES2UG24CS667>start transaction;
Query OK, 0 rows affected (0.01 sec)

PES2UG24CS660_PES2UG24CS667>SAVEPOINT before_allocation_test;
Query OK, 0 rows affected (0.00 sec)

PES2UG24CS660_PES2UG24CS667>INSERT INTO allocation
-> (
->     donation_id,
->     charity_id,
->     allocated_quantity,
->     status
-> )
-> VALUES
-> (
->     (
->         SELECT donation_id
->         FROM food_donation
->         WHERE food_name = 'Rice and Curry'
->         LIMIT 1
->     ),
->     (
->         SELECT charity_id
->         FROM charity_or_ngo
->         WHERE name = 'Food For All Foundation'
->         LIMIT 1
->     ),
->     5.00,
->     'PENDING'
-> );
Query OK, 1 row affected (0.00 sec)

PES2UG24CS660_PES2UG24CS667>SELECT
->     fd.food_name,
->     a.allocated_quantity,
->     a.status
-> FROM allocation a
-> JOIN food_donation fd
->   ON a.donation_id = fd.donation_id
-> WHERE fd.food_name = 'Rice and Curry';

```

food_name	allocated_quantity	status
Rice and Curry	5.00	PENDING
Rice and Curry	5.00	PENDING

```

2 rows in set (0.00 sec)

```

- ROLLBACK TO SAVEPOINT
- COMMIT

```
PES2UG24CS660_PES2UG24CS667>ROLLBACK TO SAVEPOINT before_allocation_test;
Query OK, 0 rows affected (0.00 sec)
```

```
PES2UG24CS660_PES2UG24CS667>SELECT
-> fd.food_name,
-> a.allocated_quantity,
-> a.status
-> FROM allocation a
-> JOIN food_donation fd
-> ON a.donation_id = fd.donation_id
-> WHERE fd.food_name = 'Rice and Curry';
```

food_name	allocated_quantity	status
Rice and Curry	5.00	PENDING

1 row in set (0.00 sec)

```
PES2UG24CS660_PES2UG24CS667>INSERT INTO allocation
-> (
-> donation_id,
-> charity_id,
-> allocated_quantity,
-> status
-> )
-> VALUES
-> (
-> (
-> SELECT donation_id
-> FROM food_donation
-> WHERE food_name = 'Rice and Curry'
-> LIMIT 1
-> ),
-> (
-> SELECT charity_id
-> FROM charity_or_ngo
-> WHERE name = 'Food For All Foundation'
-> LIMIT 1
-> ),
-> 5.00,
-> 'PENDING'
-> );
```

```
Query OK, 1 row affected (0.00 sec)
```

```
PES2UG24CS660_PES2UG24CS667>COMMIT;
Query OK, 0 rows affected (0.01 sec)
```

```
PES2UG24CS660_PES2UG24CS667>SELECT
-> fd.food_name,
-> a.allocated_quantity,
-> a.status
-> FROM allocation a
-> JOIN food_donation fd
-> ON a.donation_id = fd.donation_id
-> WHERE fd.food_name = 'Rice and Curry';
```

food_name	allocated_quantity	status
Rice and Curry	5.00	PENDING
Rice and Curry	5.00	PENDING

2 rows in set (0.00 sec)

```
PES2UG24CS660_PES2UG24CS667>
```

12. CURSORS

12.1 CURSORS FOR PROCESSING FOOD DONATIONS

```

PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>CREATE PROCEDURE ProcessFoodDonations()
-> BEGIN
->     DECLARE done INT DEFAULT 0;
->
->     DECLARE v_food_name VARCHAR(255);
->     DECLARE v_quantity DECIMAL(10,2);
->     DECLARE v_status VARCHAR(50);
->
->     DECLARE donation_cursor CURSOR FOR
->         SELECT food_name, quantity, status
->         FROM food_donation;
->
->     DECLARE CONTINUE HANDLER FOR NOT FOUND
->         SET done = 1;
->
->     OPEN donation_cursor;
->
->     read_loop: LOOP
->
->         FETCH donation_cursor
->         INTO v_food_name, v_quantity, v_status;
->
->         IF done = 1 THEN
->             LEAVE read_loop;
->         END IF;
->
->         SELECT
->             v_food_name AS food_name,
->             v_quantity AS quantity,
->             v_status AS status;
->
->     END LOOP;
->
->     CLOSE donation_cursor;
-> END $$

```

Query OK, 0 rows affected (0.01 sec)

```

PES2UG24CS660_PES2UG24CS667>
PES2UG24CS660_PES2UG24CS667>DELIMITER ;
PES2UG24CS660_PES2UG24CS667>CALL ProcessFoodDonations();

```

food_name	quantity	status
Vegetable Biryani	50.00	AVAILABLE

1 row in set (0.01 sec)

food_name	quantity	status
Fresh Bread	30.00	AVAILABLE

1 row in set (0.01 sec)

food_name	quantity	status
Rice and Curry	75.00	AVAILABLE

1 row in set (0.01 sec)

Query OK, 0 rows affected (0.01 sec)

PES2UG24CS660_PES2UG24CS667>

13. COMMON TABLE EXPRESSIONS

13.1 CTE FOR DONATION ANALYSIS AND WINDOW FUNCTION RANKING

```

PES2UG24CS660_PES2UG24CS667>WITH donation_summary AS (
-> SELECT
->     fd.donation_id,
->     fd.food_name,
->     fd.quantity,
->     COALESCE(SUM(a.allocated_quantity), 0) AS allocated_quantity
-> FROM food_donation fd
-> LEFT JOIN allocation a
->     ON fd.donation_id = a.donation_id
-> GROUP BY
->     fd.donation_id,
->     fd.food_name,
->     fd.quantity
-> )
-> SELECT
->     food_name,
->     quantity AS donated_quantity,
->     allocated_quantity,
->     quantity - allocated_quantity AS remaining_quantity
-> FROM donation_summary
-> ORDER BY remaining_quantity DESC;

```

food_name	donated_quantity	allocated_quantity	remaining_quantity
Rice and Curry	75.00	10.00	65.00
Vegetable Biryani	50.00	30.00	20.00
Fresh Bread	30.00	20.00	10.00

3 rows in set (0.00 sec)

```

PES2UG24CS660_PES2UG24CS667>SELECT
->     food_name,
->     quantity,
->     RANK() OVER (ORDER BY quantity DESC) AS donation_rank
-> FROM food_donation;

```

food_name	quantity	donation_rank
Rice and Curry	75.00	1
Vegetable Biryani	50.00	2
Fresh Bread	30.00	3

3 rows in set (0.01 sec)

```

PES2UG24CS660_PES2UG24CS667>

```

13.2 ADVANCED QUERY – CORRELATED SUBQUERY AND ALLOCATION PERCENTAGE

```
PES2UG24CS660_PES2UG24CS667>SELECT
```

```
-> fd.food_name,  
-> fd.quantity,  
-> fd.location  
-> FROM food_donation fd  
-> WHERE fd.quantity > (  
-> SELECT AVG(fd2.quantity)  
-> FROM food_donation fd2  
-> );
```

food_name	quantity	location
Rice and Curry	75.00	Bangalore

```
1 row in set (0.00 sec)
```

```
PES2UG24CS660_PES2UG24CS667>SELECT
```

```
-> d.name AS donor_name,  
-> COUNT(fd.donation_id) AS total_donations,  
-> COALESCE(SUM(fd.quantity), 0) AS total_quantity  
-> FROM food_donor d  
-> LEFT JOIN food_donation fd  
-> ON d.donor_id = fd.donor_id  
-> GROUP BY d.donor_id, d.name  
-> ORDER BY total_quantity DESC;
```

donor_name	total_donations	total_quantity
Annapurna Caterers	1	75.00
Green Leaf Restaurant	1	50.00
FreshMart Supermarket	1	30.00
Daily Meals Restaurant	0	0.00

```
4 rows in set (0.00 sec)
```

```
PES2UG24CS660_PES2UG24CS667>SELECT
```

```
-> fd.food_name,  
-> fd.quantity AS donated_quantity,  
-> COALESCE(SUM(a.allocated_quantity), 0) AS allocated_quantity,  
-> ROUND(  
-> COALESCE(SUM(a.allocated_quantity), 0)  
-> * 100 / fd.quantity,  
-> 2  
-> ) AS allocation_percentage  
-> FROM food_donation fd  
-> LEFT JOIN allocation a  
-> ON fd.donation_id = a.donation_id  
-> GROUP BY  
-> fd.donation_id,  
-> fd.food_name,  
-> fd.quantity  
-> ORDER BY allocation_percentage DESC;
```

food_name	donated_quantity	allocated_quantity	allocation_percentage
Fresh Bread	30.00	20.00	66.67
Vegetable Biryani	50.00	30.00	60.00
Rice and Curry	75.00	10.00	13.33

```
3 rows in set (0.01 sec)
```

14. FINAL VERIFICATION & PROJECT COMPLETION

14.1 Verification of database tables and viwe tables

```
PES2UG24CS660_PES2UG24CS667>SHOW TABLES;
+-----+
| Tables_in_ecofoodsync_db |
+-----+
| allocation
| charity_or_ngo
| food_donation
| food_donor
| food_request
| pickup_delivery
| volunteer
| vw_active_deliveries
+-----+
8 rows in set (0.00 sec)

PES2UG24CS660_PES2UG24CS667>SHOW FULL TABLES
-> WHERE TABLE_TYPE = 'VIEW';
+-----+-----+
| Tables_in_ecofoodsync_db | Table_type |
+-----+-----+
| vw_active_deliveries      | VIEW       |
+-----+-----+
1 row in set (0.00 sec)

PES2UG24CS660_PES2UG24CS667>|
```

14.2 Verification of Stored Procedures and functions

```
PES2UG24CS660_PES2UG24CS667>SHOW PROCEDURE STATUS
-> WHERE Db = DATABASE();
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Db      | Name                                     | Type      | Definer      | Modified      | Created      | Security_type | Comment      | character_set_client | collation_connection | Database Collation |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ecofoodsync_db | GetAvailableFoodByLocation             | PROCEDURE | root@localhost | 2026-10-04 08:30:52 | 2026-10-04 08:30:52 | DEFINER      |              | cp850              | cp850_general_ci    | utf8mb4_0900_ai_ci |
| ecofoodsync_db | GetDonationAllocationSummary           | PROCEDURE | root@localhost | 2026-10-04 08:30:52 | 2026-10-04 08:30:52 | DEFINER      |              | cp850              | cp850_general_ci    | utf8mb4_0900_ai_ci |
| ecofoodsync_db | ProcessFoodDonations                   | PROCEDURE | root@localhost | 2026-10-04 09:19:38 | 2026-10-04 09:19:38 | DEFINER      |              | cp850              | cp850_general_ci    | utf8mb4_0900_ai_ci |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
3 rows in set (0.00 sec)

PES2UG24CS660_PES2UG24CS667>SHOW FUNCTION STATUS
-> WHERE Db = DATABASE();
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Db      | Name                                     | Type      | Definer      | Modified      | Created      | Security_type | Comment      | character_set_client | collation_connection | Database Collation |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ecofoodsync_db | GetRemainingDonationQuantity           | FUNCTION  | root@localhost | 2026-10-04 08:30:53 | 2026-10-04 08:30:53 | DEFINER      |              | cp850              | cp850_general_ci    | utf8mb4_0900_ai_ci |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)

PES2UG24CS660_PES2UG24CS667>|
```

```
PES2UG24CS660_PES2UG24CS667>SHOW TRIGGERS;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| Trigger      | Event      | Table      | Statement
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| t_client      | collation_connection | Database Collation | Timing | Created      | sql_node      | Definer      | character_so
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| before_allocation_insert | INSERT | allocation | BEGIN
| DECLARE available_qty DECIMAL(10,2);
| SELECT
|   quantity -
|   COALESCE(
|     (
|       SELECT SUM(allocated_quantity)
|       FROM allocation
|       WHERE donation_id = NEW.donation_id
|     ),
|     0
|   )
| INTO available_qty
| FROM food_donation
| WHERE donation_id = NEW.donation_id;
| IF NEW.allocated_quantity > available_qty THEN
|   SIGNAL SQLSTATE '45000'
|   SET MESSAGE_TEXT =
|     'Allocation exceeds available donation quantity';
| END IF;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
END | BEFORE | 2026-10-04 08:30:53.07 | ONLY_FULL_GROUP_BY, STRICT_TRANS_TABLES, NO_ZERO_IN_DATE, NO_ZERO_DATE, ERROR_FOR_DIVISION_BY_ZERO, NO_ENGINE_SUBSTITUTION | root@localhost | cp850 | cp850_general_ci | utf8mb4_0900_ai_ci |
| before_food_donation_insert | INSERT | food_donation | BEGIN
| IF NEW.expiry_time <= NEW.preparation_time THEN
|   SIGNAL SQLSTATE '45000'
|   SET MESSAGE_TEXT =
|     'Expiry time must be after preparation time';
| END IF;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
END
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
ocalhost | cp850 | cp850_general_ci | utf8mb4_0900_ai_ci | BEFORE | 2026-10-04 08:30:53.09 | ONLY_FULL_GROUP_BY, STRICT_TRANS_TABLES, NO_ZERO_IN_DATE, NO_ZERO_DATE, ERROR_FOR_DIVISION_BY_ZERO, NO_ENGINE_SUBSTITUTION | root@l
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

PES2UG24CS660_PES2UG24CS667>|
```

14.3 VERIFY INDEX

```
PES2UG24CS660_PES2UG24CS667>SHOW INDEX FROM food_donation;
```

Table	Non_unique	Key_name	Seq_in_index	Column_name	Collation	Cardinality	Sub_part	Packed	Null	Index_type	Comment	Index_comment	Visible	Expression
food_donation	0	PRIMARY	1	donation_id	A	3	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_food_donation_donor	1	donor_id	A	3	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_donation_status_expiry	1	status	A	1	NULL	NULL	YES	BTREE			YES	NULL
food_donation	1	idx_donation_status_expiry	2	expiry_time	A	3	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_donation_status_expiry	3	food_condition	A	3	NULL	NULL		BTREE			YES	NULL
food_donation	1	idx_food_name_fulltext	1	food_name	NULL	3	NULL	NULL		FULLTEXT			YES	NULL

6 rows in set (0.00 sec)

```
PES2UG24CS660_PES2UG24CS667>
```

14.4 VERIFICATION OF TRANSACTION TABLE

```
PES2UG24CS660_PES2UG24CS667>SELECT
-> fd.food_name,
-> fd.quantity AS donated_quantity,
-> COALESCE(SUM(a.allocated_quantity), 0) AS allocated_quantity,
-> fd.quantity - COALESCE(SUM(a.allocated_quantity), 0) AS remaining_quantity
-> FROM food_donation fd
-> LEFT JOIN allocation a
-> ON fd.donation_id = a.donation_id
-> GROUP BY
-> fd.donation_id,
-> fd.food_name,
-> fd.quantity
-> ORDER BY fd.food_name;
```

food_name	donated_quantity	allocated_quantity	remaining_quantity
Fresh Bread	30.00	20.00	10.00
Rice and Curry	75.00	10.00	65.00
Vegetable Biryani	50.00	30.00	20.00

3 rows in set (0.00 sec)